

High-Performance Multimodal AI Architectures: Leveraging Gemma 4, Whisper-HIP, and ROCm Optimizations for the AMD Developer Hackathon

Architectural Requirements and Evaluation Mechanics of the AMD ACT II Hackathon

Developing a winning solution for the AMD Developer Hackathon (ACT II) requires navigating strict operational constraints, hardware limits, and evaluation metrics.¹ The competition is structured into three specialized tracks, each demanding a distinct architectural approach to optimize performance, accuracy, and token efficiency on AMD compute resources.¹

Hackathon Track Matrix

The execution requirements across the three competition tracks show distinct design challenges, forcing a trade-off between local compute utilization and external API routing.¹

Track	Core Objective	Primary Submissions	Runtime Constraints	Evaluation Metrics
Track 1: General-Purpose AI Agent ¹	Build an autonomous agent to execute text tasks across eight capability domains. ¹	Docker Container pushed to a public registry (linux/amd64 manifest). ¹	Maximum runtime: 10 minutes. ¹ Container must startup within 60 seconds. ¹	Accuracy Gate: Evaluated by an LLM-Judge. ¹ Token Efficiency: Submissions passing accuracy are ranked ascending by total tokens. ¹
Track 2: Video	Generate	Docker	Maximum	Caption

Captioning Agent ¹	stylistic captions (formal, sarcastic, humorous tech, humorous non-tech) for video clips. ¹	Container pushed to a public registry (linux/amd64 manifest). ¹	runtime: 10 minutes. ¹ Response time per request under 30 seconds. ¹	Accuracy (0-1): Visual fidelity. ¹ Style Match (0-1): Tone alignment. ¹ Weighted average score across ~12 hidden clips. ¹
Track 3: Unicorn (Open Innovation) ¹	Develop an original, practically useful AI application leveraging AMD compute infrastructure. ¹	Public GitHub Repository, Demo Video, Slide Deck (PDF), and a Live Demo URL. ¹	Dependent on hosted cloud VM or local setup. ⁴	Checked for AMD compute usage (disqualified if absent) ¹ , followed by human judging. ¹

Baselines and Target Domains for Video Captioning

Track 2 requires a high degree of generalization across varied video content.¹ While three baseline clips are provided for development, the evaluation set spans multiple unconstrained visual domains.¹

Clip ID	Target URL	Visual Content	Domain Challenge
v1 ¹	https://storage.googleapis.com/amd-hackathon-clips/1860079-uhd_2560_1440_25fps.mp4 ¹	Urban autumn boulevard, golden trees, moving city traffic. ¹	Motion tracking, environmental light changes, background vehicle clutter. ¹
v2 ¹	https://storage.googleapis.com/amd-hackathon-clips/13825391-uhd_3840_2160_30fps.mp4 ¹	Orange kitten playing among green foliage in a garden. ¹	High-frequency spatial details, fast organic movement, occlusion by plants. ¹

v3 ¹	https://storage.googleapis.com/amd-hackathon-clips/3044693-uhd_3840_2160_24fps.mp4 ¹	Office worker typing at a desktop in a modern open-plan layout. ¹	Fine-grained activity recognition, text on screens, static ambient conditions. ¹
Hidden Set ¹	Undisclosed (~12 clips, 30 seconds to 2 minutes in length). ¹	Nature, urban scenery, animals, human actions, sports, food, weather, technology. ¹	Multi-event segmentation, acoustic variance, speech-visual alignment, domain adaptation. ¹

Evaluation Optimization Formulations

To maximize the final leaderboard position in Track 1, the agent must minimize token utilization while remaining above a designated accuracy threshold.¹ This routing optimization problem can be formulated mathematically. Let T be the set of tasks, $a_t(m)$ be the accuracy of model m on task t , and $c_t(m)$ be the token cost of model m on task t .¹ The routing policy must select a model $m \in \{\text{local}, \text{remote}\}$ for each task t to minimize total token cost while satisfying the global accuracy gate¹:

$$\text{Minimize } \sum_{t \in T} \mathbb{I}(m_t = \text{remote}) \cdot c_t(\text{remote})$$

$$\text{Subject to } \frac{1}{|T|} \sum_{t \in T} a_t(m_t) \geq \tau_{\text{accuracy}}$$

where $\mathbb{I}$ is the indicator function, τ_{accuracy} is the LLM-Judge accuracy gate, and local tokens are calculated as zero cost.¹

For Track 2, the scoring objective is a weighted average of caption accuracy and style

matching.¹ Let V represent the hidden evaluation video set, S represent the four target

styles, $A(v, s)$ be the visual accuracy score for video v in style s , and $M(v, s)$ be the style match score.¹ The objective maximizes the final joint score¹:

$$\text{Maximize } \Phi = \frac{1}{|V| \cdot |S|} \sum_{v \in V} \sum_{s \in S} (\alpha \cdot A(v, s) + \beta \cdot M(v, s))$$

where α and β are the scoring weights allocated to content accuracy and style fidelity, respectively.¹

Input and Output I/O Interfaces

For the agent to run successfully in the evaluation harness, it must process inputs and write outputs using strict JSON formatting.¹ The input harness maps tasks through /input/tasks.json, and the container must write to /output/results.json before exiting with code 0.¹

For Track 1, the input format lists task IDs and raw prompts¹:

JSON

The resulting output must write to the target directory containing the generated answer string¹:

JSON

For Track 2, the agent is provided with target video URLs and the list of styles to generate¹:

JSON

```
[
  {
    "task_id": "v1"
    "video_url":
    "https://storage.googleapis.com/amd-hackathon-clips/1860079-uhd_2560_1440_25fps.mp4"
    "styles": "formal" "sarcastic" "humorous_tech" "humorous_non_tech"
  }
]
```

The output JSON must contain captions for all requested styles, as any missing style defaults to a score of zero for that clip ¹:

JSON

SOTA Video Captioning Architectures and Multimodal Fusion Systems

Developing a competitive video captioning pipeline requires moving past naive visual-only approaches. ⁶ Standard paradigms often fail to capture temporal boundaries and cross-modal cues because they treat video as a flat sequence of silent frames. ⁵ High-performance architectures resolve these limitations through structured multi-stream factorization and temporal localization.

Multi-Stream Scene Script (MTSS)

The Multi-Stream Scene Script (MTSS) framework addresses the limits of flat visual sequences by factorizing video into four specialized, traceably connected streams ⁸:

- **Shot Stream:** Segments the video temporally into visual blocks, applying Shot-Aware Structured Attention to prevent visual feature leakage across camera cuts. ⁸
- **Event Stream:** Tracks continuous auditory transitions and localized acoustic events, capturing sound characteristics alongside spoken content. ⁸
- **Reference Stream:** Manages persistent visual identities across the video, mapping them to explicit reference tokens to maintain identity consistency. ⁸
- **Global Stream:** Captures high-level ambient context, such as lighting, location, and global motion dynamics. ⁸

This explicit decomposition isolates visual and auditory identities, preventing the visual-semantic binding errors common in monolithic prompts. ⁸

Continuous Temporal Localization via Neural ODEs

Traditional video captioning systems struggle to detect event boundaries in multi-event or untrimmed videos, relying on rigid, pre-defined segment proposals. ⁵ State-of-the-art frameworks solve this by modeling the temporal progression of visual features as a continuous trajectory using Neural Ordinary Differential Equations (Neural ODEs). ⁵

The continuous state transitions of a video sequence $\mathbf{z}(t)$ are calculated using a neural

network f parameterized by θ ⁵:

$$\frac{dz(t)}{dt} = f(z(t), t, \theta)$$

By treating temporal transitions as a continuous function, Neural ODEs identify event boundaries with high precision and low computational overhead compared to recurrent neural models.⁵

Multi-Modal Feature Fusion and Cross-Modal Retrieval

When processing complex videos, integrating acoustic and visual features is necessary to capture both spoken descriptions and visual context.⁶ The core process of a multi-modal fusion architecture is structured into three distinct stages:



First, features are extracted from their respective streams.⁶ Visual features are mapped using spatial-temporal architectures (such as the I3D network, which captures optical flow), while acoustic features are extracted using VGGish to map audio signals into text-aligned vectors.⁶ Because these vectors have different dimensions, they cannot be directly concatenated.⁶ The

multi-modal attention fusion module uses a Transformer encoder's multi-head attention to align and fuse these representations, resolving differences in sequence length.⁶ A confidence module then filters out background noise, sending only high-confidence multi-modal features to the decoder for text generation.⁶

The Gemma 4 Multimodal Family: Architecture, Hardware Suitability, and Tokenizer Constraints

Google DeepMind's Gemma 4 family, released on March 31, 2026, represents a major advance in local, open-weight multimodal models.⁹ Built on a native multimodal layout, Gemma 4 processes text, image, and audio inputs within a single unified network, avoiding the latency and alignment issues of dual-encoder systems.¹¹

Gemma 4 Model Suite Specifications

The Gemma 4 family spans several parameter sizes and architectures to balance local compute constraints against generation quality.⁹

Model Name	Architecture Type	Effective Parameters	Active Parameters	Context Window	Native Modalities	Target Deployment
Gemma 4 E2B ¹³	Dense with PLE ¹³	~ ¹⁵	~ ¹⁵	128 K ¹³	Text, Image, Audio ¹⁵	Mobile, low-power edge laptops, IoT. ⁹
Gemma 4 E4B ¹³	Dense with PLE ¹³	~ ¹⁵	~ ¹⁵	128 K ¹³	Text, Image, Audio ¹⁵	High-end mobile, workstation edge servers. ⁹
Gemma 4 26B A4B ¹³	Mixture of Experts ¹³	26 B ¹³	~ ¹⁰	256 K ¹³	Text, Image ¹⁵	Workstations, cost-sensitive high-throughput servers. ⁹

Gemma 4 31B ¹³	Dense ¹⁵	30.7 B ¹⁵	30.7 B ¹⁵	256 K ¹³	Text, Image ¹⁵	Datacenters, quality-first fine-tuning workflows. ¹⁰
----------------------------------	---------------------	--------------------------------	--------------------------------	-------------------------------	---------------------------	---

Core Architectural Innovations

Gemma 4 introduces key architectural optimizations designed to improve resource efficiency and performance on local hardware.⁹

Alternating Layered Attention

The model alternates between local sliding-window attention and global full-context attention across its layers.¹⁴ The smaller dense models use a 512-token sliding window, while the larger variants scale to 1024 tokens.¹⁴ This design limits the quadratic complexity of standard attention mechanisms, maintaining long-range context tracking while reducing computing requirements.¹⁴

Per-Layer Embeddings (PLE)

The on-device models (E2B and E4B) incorporate Per-Layer Embeddings (PLE) to bypass the information bottleneck of traditional shared embeddings.¹³ Instead of passing a single input token vector through all subsequent layers, PLE feeds an auxiliary residual signal directly into each individual decoder layer.¹⁴

The layer-specific input vector $\mathbf{x}_{\text{layer}}$ is defined by mixing the core token identity with context-aware visual or acoustic projections¹⁴:

$$\mathbf{x}_{\text{layer}} = \frac{1}{\sqrt{2}} (\mathbf{t}_{\text{identity}} + \mathbf{c}_{\text{context-aware}})$$

This mechanism provides visual and auditory tracking for each transformer block without inflating the global model footprint.¹⁴

Shared KV Cache and Dual RoPE Configurations

To reduce memory footprint during long-context generation, Gemma 4 reuses Key and Value projections from the last non-shared layer of the same attention type in its final decoder blocks.¹⁴ To stabilize context scaling up to **256 K** tokens, local sliding-window layers apply standard RoPE, while global full-attention layers deploy scaled proportional RoPE (p-RoPE).¹³

Encoder-Free Native Multimodality

The Gemma 4 12B model features an encoder-free architecture.¹¹ Standard multimodal models

rely on separate pretrained encoders to process images and audio, which can lead to alignment errors.¹¹ Gemma 4 12B projects raw visual patches and raw acoustic waveforms directly into the main LLM's embedding space through simple linear projection layers, relying entirely on the core language model for multimodal reasoning.¹¹

The E2B and E4B models retain compact encoders—a $\sim 150 \text{ M}$ parameter vision encoder and a $\sim 300 \text{ M}$ parameter USM-style Conformer audio encoder—to preserve high accuracy on resource-constrained edge platforms.¹⁴

Multi-Modal Prompting and Tokenizer Constraints

Processing multimodal datasets with Gemma 4 requires adhering to strict structural and formatting rules.¹⁶

- **Sequential Modality Ordering:** To maintain semantic alignment, visual tokens must be positioned *before* the text prompt, while audio tokens must be placed *after* the text prompt.¹⁶
- **Resolution and Patch Alignments:** Visual inputs must have dimensions divisible by 48 ($48 = \text{patch size } 16 \times \text{pooling kernel } 3$).¹⁹ Image inputs are scaled and cropped to fit a specific visual token budget, which varies from 70 to 1120 tokens depending on the target accuracy and speed constraints.¹⁴
- **Thinking Mode Protocol:** Gemma 4 models feature an explicit chain-of-thought reasoning mode.¹⁰ This is triggered by appending the `<|think|>` token to the system prompt.¹⁶ The model then outputs its step-by-step reasoning within `<|channel|>thought\n` and `<channel|>` tags before generating the final answer.¹⁶ In multi-turn templates, historical reasoning blocks must be stripped from the context window to prevent repetition errors.¹⁶
- **Special Token Exclusions:** Standard text prompts must not contain explicit visual tokens like `<|image|>`, `<img>`, or `<image_soft_token>`, as these trigger tokenization errors in the processor.²² Instead, image placeholders are handled by the system chat template.²²

Hugging Face Implementation Blueprint

The Hugging Face Transformers library supports loading and executing Gemma 4's multimodal capabilities.²² For video captioning, the pipeline can process video frames and extract acoustic streams natively.²¹

```
Python
```

```
import
```

```

from transformers import AutoProcessor, AutoModelForMultimodalLM

# Load the target Gemma 4 model with automatic device mapping
model_id = "google/gemma-4-E4B-it"
processor = AutoProcessor.from_pretrained(model_id)
model = AutoModelForMultimodalLM.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16,
    device_map="auto"
)

# Structure the multimodal chat message list
# Notice that visual frames are placed before the text, and audio is placed after the text
messages = [
    {
        "role": "user",
        "content": [
            {"type": "image", "image": "frame_01.png"},
            {"type": "image", "image": "frame_02.png"},
            {"type": "text", "text": "Analyze the visual sequence and transcribe the associated speech."},
            {"type": "audio", "audio": "extracted_audio.wav"}
        ]
    }
]

# Apply chat template and process inputs
# Setting load_audio_from_video=True leverages native pipeline extraction
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    return_dict=True,
    return_tensors="pt",
    add_generation_prompt=True,
    load_audio_from_video=True
).to(model.device)

# Generate tokens using the recommended sampling configuration
generation_outputs = model.generate(
    *inputs,
    max_new_tokens=512,
    temperature=1.0,
    top_p=0.95,
    top_k=64
)

```

```
# Extract and decode generated text
input_length = input["input_ids"].shape[1]
response_tokens = generation_outputs[input_length:]
rtnnprocessor.decode(response_tokens, skip_special_tokens=True)
```

High-Performance Speech-to-Text and Computer Vision Pipelines on AMD Hardware

When deploying local models within Hackathon containers, developers must optimize the performance of the underlying computer vision and speech-to-text components.¹ This involves compiling execution libraries to match specific AMD GPU architectures and managing memory to prevent Out-of-Memory (OOM) faults.²⁵

CTranslate2 Compilation and WHISPER-HIP Optimization

In February 2026, CTranslate2 merged native support for AMD Radeon and Instinct GPUs via the HIP runtime.²⁶ This allows the faster-whisper package to run with full hardware acceleration on AMD platforms.²⁶

To build CTranslate2 with AMD GPU support, the environment must compile the C++ execution engine to match the specific graphics processor version (gfx ID).²⁶

GPU Model Name	Architecture ID (gfx target)	Performance Profile	Quantization Alignment
AMD Instinct MI300X ⁴	gfx942 ²⁶	High-throughput datacenter APU, 192 GB HBM3 memory bandwidth. ⁴	FP8 GEMM, native INT8, FP16 execution. ³¹
AMD Radeon RX 7900 XTX ³³	gfx1100 ²⁶	High-performance workstation card, 24 GB GDDR6 memory. ³³	FP16, INT8, Q4_K_M GGUF format. ³³
AMD Radeon RX 6600 ²⁵	gfx1032 ²⁶	Mid-range consumer graphics card, 8 GB	Quantized GGML (Q5_0), INT8 quantization. ²⁵

		GDDR6 memory. ²⁵	
AMD Ryzen AI (780M / 890M) ³⁴	gfx1103 / gfx1150 ³⁴	Integrated laptop processor APU, shares system memory. ³⁴	ONNX Runtime via DirectML, 4-bit quantization. ³⁰

To build the wheel for a specific architecture, the target GPU target must be exported as an environment variable before compiling:

```
Bash
# Set compilation target for AMD Radeon RX 7900 XTX
export PYTORCH_ROCM_ARCH=gfx1100
```

```
# Install CTranslate2-HIP wheel from the release page
```

```
pip install ctranslate2_rocm-4.7.1-cp310-cp310-linux_x86_64.whl
```

```
pip install faster-whisper
```

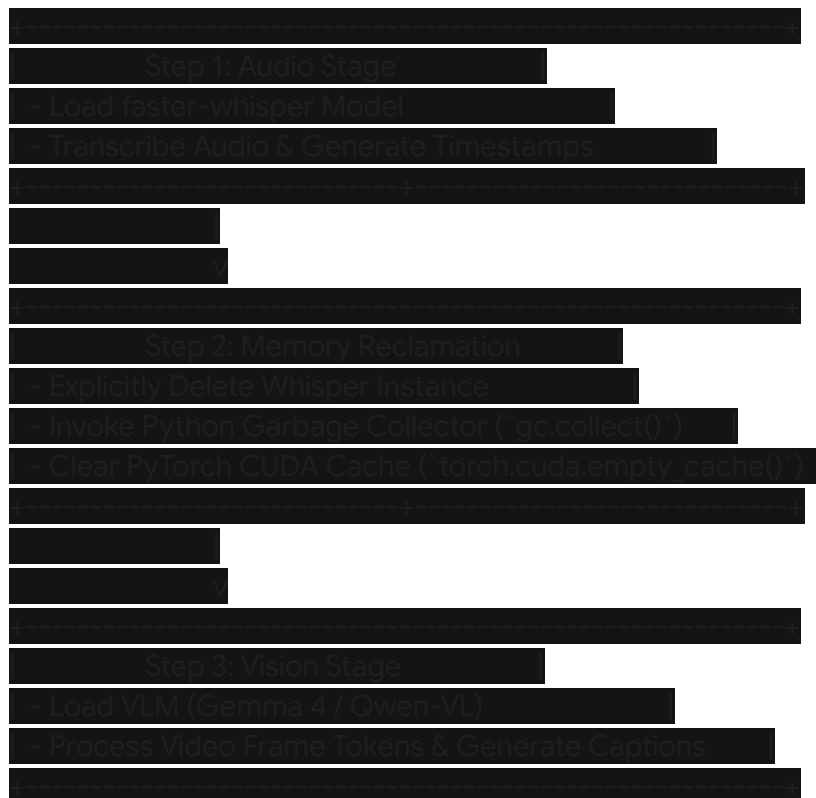
For cards not officially listed in the ROCm support matrix, developers can override the target version check²⁵:

```
Bash
# Force ROCm to run on unvalidated GFX cards
export HSA_OVERRIDE_GFX_VERSION=10.3.0 # For gfx1032 / RX 6600
export CT_CUDA_ALLOCATOR=cub_caching # Prevent fragmentation on RDNA2
```

VRAM Optimization via Sequential Execution

Containers deployed in the hackathon have a strict size limit of **10 GB** and must run within defined VRAM constraints.¹ Attempting to load a Whisper speech model and a Vision-Language Model (VLM) into VRAM simultaneously often causes immediate OOM crashes on systems with **8 GB** to **16 GB** of memory.²⁷ To prevent this, the pipeline must implement a sequential

model loading workflow.²⁷



This sequence ensures that the model weights of each stage are cleanly evicted from memory before the next model is loaded, allowing large models to run successfully on resource-constrained host machines.²⁷

Computer Vision Optimization: Keyframe Extraction

Naively processing every frame of a 30-second to 2-minute video through a VLM is slow and consumes massive amounts of visual tokens, easily exceeding the model's context window.²⁰

To optimize visual processing, the pipeline should implement scene-change detection to extract only representative keyframes.²⁷

```

Python
import cv2
import numpy as np
from PIL import Image

def extract_representative_keyframes(video_path: str, threshold: float = 30.0) -> list:
    """
    Extracts representative keyframes from a video based on pixel-level variance
    to detect scene cuts, avoiding visual redundancy and saving token budget.
    """
    cap = cv2.VideoCapture(video_path)
    keyframes = []
    prev_frame = None

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        # Convert frame to grayscale and downsample to accelerate comparison
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        resized = cv2.resize(gray, 100, 100)

        if prev_frame is None:
            # Always capture the initial frame as the baseline keyframe
            keyframes.append(Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)))
            prev_frame = resized
            continue

        # Compute the absolute difference between successive frames
        diff = cv2.absdiff(resized, prev_frame)
        mean_diff = np.mean(diff)

        if mean_diff > threshold:
            # Keyframe detected due to substantial visual change
            keyframes.append(Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)))
            prev_frame = resized

    cap.release()
    return keyframes

```

This approach limits input tokens by filtering out redundant, identical frames, speeding up

processing and keeping the input within the model's context limits.²⁰

Advanced Prompt Engineering and Persona Simulation

Generating stylistic captions for Track 2 requires the model to adopt distinct, authentic personas.¹ Standard LLM prompts often produce flat, overly polite, or homogenized responses due to safety and alignment training.³⁵ To force the model to adopt the requested styles, the pipeline must employ advanced prompting techniques and persona simulation.

Pragmatic Metacognitive Prompting (PMP) for Sarcasm

Sarcasm requires detecting structural contrasts between what is visible and what is expected.³⁷ Standard prompts often generate overly positive or generic puns.³⁸ To generate sharp, dry irony, the system uses Pragmatic Metacognitive Prompting (PMP) to guide the model through a step-by-step contrast analysis before it writes the caption.³⁷



This multi-step reasoning process prevents the model from falling back on safe, polite responses, producing sharper, more context-aware sarcasm.³⁶

Stylistic System Prompts and Architectures

To ensure consistent performance across the four target styles, the system prompt must explicitly define the persona boundaries, vocabulary rules, and structural formatting for each style.¹

Formal Style Prompting

The formal style must be objective, professional, and factual.¹ The prompt must restrict subjective adjectives, compliments, or conversational filler.³⁵

```
You are an expert, objective video archivist. Your task is to analyze the visual sequence and generate a factual, professional description of the events. Follow these rules:  
1. Write strictly in the objective, third-person perspective.  
2. Structure the caption using the inverted pyramid format, placing the primary event details in the opening clause.  
3. Completely avoid subjective adjectives, emotional interpretations, or flattery.  
4. Utilize precise, industry-appropriate terminology to describe objects, actions, and environmental conditions.
```

Sarcastic Style Prompting

The sarcastic style must be dry and mocking without relying on cheesy puns.¹ To bypass standard alignment restrictions, the prompt directs the model to adopt a critical, unimpressed persona.³⁵

```
You are a highly cynical, grumpy critic who is deeply unimpressed by everything. Your task is to describe the video by highlighting its pointlessness, banality, or minor flaws. Follow these rules:  
1. Maintain a dry, ironic, and biting tone.  
2. Identify obvious contradictions between human effort and the triviality of the outcome.  
3. Do not use generic puns or wacky, overly happy humor. Be sharp, pessimistic, and
```


Dismissive

4. If a scene appears standard, describe it as if it is a monumental waste of planetary energy.

Humorous Tech Style Prompting

The humorous tech style must combine technical accuracy with developer-focused humor.¹ It targets common software engineering pain points, such as legacy code, API failures, git conflicts, and the confusion of "vibe coding".⁴⁰

You are a veteran DevOps engineer reviewing a video sequence. Your task is to describe the scene entirely through software engineering, programming, and system administration metaphors. Follow these rules:

1. Map physical actions to programming concepts (e.g., human movement as a memory leak, a group conversation as a git merge conflict).

2. Reference specific developer struggles, such as attempting to exit Vim, dropping production databases, or treating code as a conservation area because it is full of bugs.

3. Incorporate tech culture concepts, such as "vibe coding" (blindly pasting prompts into an LLM and hoping it works).

4. Do not use generic, surface-level tech terms. Make the jokes specific to system architectures, compilers, and APIs.

Humorous Non-Tech Style Prompting

The humorous non-tech style targets light, everyday humor.¹ It focuses on cringey puns, observational dad jokes, and exaggerated descriptions of ordinary situations.³⁸

You are an observational stand-up comedian. Your task is to describe the video using light, everyday humor. Follow these rules:

1. Avoid any programming terms, developer jargon, or complex science concepts.

2. Use classic observational comedy tropes (e.g., comparing a subject's behavior to everyday embarrassments).

3. Integrate clean, relatable dad-joke puns (e.g., parallel lines having so much in common).

```
ut never meeting.
```

```
Exaggerate the emotional states and motivations of the subjects for comedic effect.
```

Cloud Scaling and Serving Optimization on AMD Instinct MI300X Infrastructure

For Hackathon Track 3, projects must demonstrate meaningful integration with AMD hardware.¹ Scaling multimodal models (such as Gemma 4 31B or large vision-language models) requires utilizing optimized serving frameworks and hardware-specific configurations on AMD Instinct MI300X GPUs.²⁹

AMD Instinct MI300X System Configuration

The AMD Instinct MI300X platform provides massive memory capacity, featuring **192 GB** of High Bandwidth Memory (HBM3) and up to **5.3 TB/s** of peak memory bandwidth per GPU.⁴ This large footprint allows developers to host large models and allocate substantial memory to the KV cache, which prevents out-of-memory faults during long-context generation.³¹ To run heavy workloads, the environment must disable NUMA auto-balancing to prevent latency spikes during host-to-device memory copy operations³¹:

```
Bash
```

```
# Disable kernel-level NUMA balancing
```

```
sudo systemctl --w kernel.numa_balancing
```

vLLM serving on ROCm

The ROCm-optimized fork of vLLM leverages high-performance custom attention kernels, such as Composable Kernel FlashAttention-2, to maximize throughput on CDNA3 architectures.³¹

```
Bash
```

```
# Pull the optimized ROCm-vLLM container
```

```
docker pull vllm/vllm-openai-rocm-deeplma
```

```
# Spin up Gemma 4 31B Instruct with Triton attention backend
```

```
llm serve google/gemma-4-31B-instruct
```

```
--attention-backend TRITON --ATTN
```

```
--kv-cache-dtype fp8
```

```
--max-model-len 32768
```

To maximize serving throughput, the environment must configure several low-level performance variables ³²:

- export HIP_FORCE_DEV_KERNARG=1: Bypasses host-side overhead by forcing kernel launch arguments to live directly on-device.³²
- export TORCH_BLAS_PREFER_HIPBLASLT=1: Directs PyTorch to use hipBLASLt over standard libraries, optimizing General Matrix Multiply (GEMM) operations.⁴⁶
- export SAFETENSORS_FAST_GPU=1: Enables parallel multi-threaded GPU-accelerated weight loading, cutting model initialization times down to seconds.⁴⁶
- export NCCL_MIN_NCHANNELS=112: Increases the number of communication channels for multi-GPU distributed workloads, optimizing xGMI interconnect scaling.³²

Quantization and Performance Metrics

The Instinct MI300X often outperforms competitor platforms in memory-bound scenarios, such as the decoding phase of generative inference.³¹

Using FP8 quantization (`--kv-cache-dtype fp8`) compresses the memory footprint of the Key-Value cache.³² This allows the system to process larger batch sizes and extra-long sequence lengths, alleviating KV cache eviction bottlenecks.³¹

Furthermore, deploying Multi-Token Prediction (MTP) drafters for speculative decoding allows the target model to verify multiple draft tokens in parallel, significantly accelerating structured generation tasks.¹¹

Strategic Architectures and Implementation Blueprints for Victory

To maximize the chance of winning the AMD ACT II Hackathon, developers should implement the following highly optimized architectures.

Track 2 Winner Blueprint: The Dual-Model Hybrid Captioning Pipeline

To win Track 2 under the strict 10-minute container limit, the agent must balance visual precision against stylistic generation quality.¹ Naive visual-only pipelines miss auditory cues and struggle to match complex styles.⁶ The recommended solution implements a **Dual-Model Hybrid Captioning Pipeline**, decoupling speech transcription from visual analysis to manage memory efficiently.²⁷





This sequence optimizes resource utilization ²⁷:

1. **Ingestion Stage:** The pipeline reads the video source, extracts the raw audio track using an optimized ffmpeg subprocess, and converts it to a mono **16 kHz** WAV stream.²⁰
2. **Audio Stage:** The pipeline loads faster-whisper using CTranslate2-HIP, targeting the system's specific graphics card version.²⁶ It transcribes the audio, generating a clean text transcript with precise word-level timestamps.²⁷
3. **Memory Reclamation:** Once transcription is complete, the pipeline deletes the Whisper instance and invokes torch.cuda.empty_cache(), freeing the VRAM budget for the next stage.²⁷
4. **Vision Stage:** A lightweight OpenCV subprocess extracts representative keyframes based on pixel-level variance to detect scene cuts, and aligns them with the corresponding transcript timestamps.²⁷
5. **Synthesis Stage:** The pipeline loads Gemma 4 E4B in 4-bit precision.⁹ To maintain semantic alignment, it constructs the prompt by placing the keyframes first, the audio transcript second, and the targeted style prompt at the end.¹⁶ This forces the model's native cross-modal attention to reconcile visual frames with spoken events, generating contextually rich captions.²⁰
6. **Output Stage:** The generated captions are validated against the requested schema and exported directly to /output/results.json.¹

Track 3 Winner Blueprint: Video-Oracle (Multimodal Semantic Search and Interactive QA)

For the open-innovation Unicorn track, projects must demonstrate innovative and practical use of AMD infrastructure.¹ The recommended project is **Video-Oracle**: an enterprise-grade multimodal semantic video search and interactive question-answering platform running on AMD Instinct MI300X cloud GPUs.²





The system operates as an end-to-end multimodal index:

- **Multimodal Vector Ingestion:** The ingestion pipeline processes video archives, extracting visual keyframes and acoustic chunks.²⁰ These components are mapped into a shared vector space utilizing high-performance embedding models.⁷
- **ROCm-Optimized Serving:** Gemma 4 31B is served using the high-throughput vLLM-ROCm framework on an AMD Instinct MI300X GPU.²⁹ Enabling FP8 KV-cache quantization allows the system to allocate substantial memory to processing large batch sizes with low latency.³¹
- **Semantic Video Retrieval:** Users can search the video corpus using natural language queries (e.g., "Find the moment the developer dropped the production database").⁴² The system runs a vector search over the index, retrieves the most relevant video segment, and extracts its keyframes and audio transcript.⁵⁰
- **Interactive Multimodal RAG:** The retrieved visual keyframes and transcript are sent to the served Gemma 4 31B model.⁵⁰ The model generates an interactive, context-aware answer referencing specific timestamps, demonstrating a complete multimodal RAG system running on AMD hardware.²

Works cited

1. Participant Guide_ AMD Developer Hackathon (ACT II).pdf
2. [Recap] AMD Developer Hackathon - Lablab.ai, accessed July 6, 2026, <https://lablab.ai/ai-hackathons/amd-developer>
3. AMD Developer Hackathon: ACT II - Lablab.ai, accessed July 6, 2026, <https://lablab.ai/ai-hackathons/amd-developer-hackathon-act-ii>

4. AMD Developer Cloud, accessed July 6, 2026,
<https://www.amd.com/en/developer/resources/cloud-access/amd-developer-cloud.html>
5. Cross-Modal Transformer-Based Streaming Dense Video Captioning with Neural ODE Temporal Localization - PMC, accessed July 6, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC11821176/>
6. Fusion of Multi-Modal Features to Enhance Dense Video Caption - MDPI, accessed July 6, 2026, <https://www.mdpi.com/1424-8220/23/12/5565>
7. Multi-Modal interpretable automatic video captioning - arXiv, accessed July 6, 2026, <https://arxiv.org/html/2411.06872v1>
8. Script-a-Video: Deep Structured Audio-visual Captions via Factorized Streams and Relational Grounding - arXiv, accessed July 6, 2026,
<https://arxiv.org/html/2604.11244v1>
9. Gemma 4: Google's New Open Model and Why It Runs Best on Consumer PCs | CORSAIR, accessed July 6, 2026,
<https://www.corsair.com/us/en/explorer/gamer/gaming-pcs/gemma-4-googles-new-open-model-and-why-it-runs-best-on-consumer-pcs/>
10. Gemma 4: Byte for byte, the most capable open models - Google Blog, accessed July 6, 2026,
<https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>
11. Introducing Gemma 4 12B: a unified, encoder-free multimodal model - Google Blog, accessed July 6, 2026,
<https://blog.google/innovation-and-ai/technology/developers-tools/introducing-gemma-4-12b/>
12. Google releases Gemma 4 under Apache 2.0 — and that license change may matter more than benchmarks | VentureBeat, accessed July 6, 2026,
<https://venturebeat.com/technology/google-releases-gemma-4-under-apache-2-0-and-that-license-change-may-matter>
13. Gemma 4 model overview | Google AI for Developers, accessed July 6, 2026,
<https://ai.google.dev/gemma/docs/core>
14. hello-rocm/docs/en/01-deploy/gemma4/gemma4_model.md at master - GitHub, accessed July 6, 2026,
https://github.com/datawhalechina/hello-rocm/blob/master/docs/en/01-deploy/gemma4/gemma4_model.md
15. Gemma 4 Complete Guide 2026, Architecture, Benchmarks, Deployment and more, accessed July 6, 2026,
<https://dev.to/aniruddhaadak/gemma-4-complete-guide-2026-architecture-benchmarks-deployment-3en9>
16. Gemma 4 - Google - Kaggle, accessed July 6, 2026,
<https://www.kaggle.com/models/google/gemma-4>
17. Gemma 4 on Cerebras—The Fastest Inference is Now Multimodal, accessed July 6, 2026,
<https://www.cerebras.ai/blog/gemma-4-on-cerebras-the-fastest-inference-is-now-multimodal>
18. Gemma 4 - Inference, Architecture, and Practical Insights - DebuggerCafe,

- accessed July 6, 2026,
<https://debuggercafe.com/gemma-4-inference-architecture-and-practical-insights/>
19. Gemma4 - Hugging Face, accessed July 6, 2026,
https://huggingface.co/docs/transformers/model_doc/gemma4
 20. Gemma 4's Audio and Video Inputs: A Hands-On Guide Nobody Has ..., accessed July 6, 2026,
<https://dev.to/pulkitgovrani/gemma-4s-audio-and-video-inputs-a-hands-on-guide-nobody-has-written-yet-2m2c>
 21. huggingface-gemma-recipes/notebooks/Gemma4_(E2B)-Multimodal.ipynb at main - GitHub, accessed July 6, 2026,
[https://github.com/huggingface/huggingface-gemma-recipes/blob/main/notebooks/Gemma4_\(E2B\)-Multimodal.ipynb](https://github.com/huggingface/huggingface-gemma-recipes/blob/main/notebooks/Gemma4_(E2B)-Multimodal.ipynb)
 22. Run Gemma with Hugging Face Transformers | Google AI for Developers, accessed July 6, 2026,
https://ai.google.dev/gemma/docs/core/huggingface_inference
 23. google/gemma-4-E4B - Hugging Face, accessed July 6, 2026,
<https://huggingface.co/google/gemma-4-E4B>
 24. ROCm - Lablab.ai, accessed July 6, 2026, <https://lablab.ai/tech/amd/amd-rocm>
 25. Running GPU-Accelerated Whisper on an AMD GPU (No NVIDIA Required) - Medium, accessed July 6, 2026,
<https://medium.com/@abhshk/running-gpu-accelerated-whisper-on-an-amd-gpu-no-nvidia-required-e27ea20b2ccd>
 26. Does faster whisper support AMD GPU? · Issue #1370 - GitHub, accessed July 6, 2026, <https://github.com/SYSTRAN/faster-whisper/issues/1370>
 27. Local Video-to-Text Pipeline on Apple Silicon (Whisper + Qwen2.5-VL) - Optimized for 8GB/16GB RAM : r/LocalLLaMA - Reddit, accessed July 6, 2026,
https://www.reddit.com/r/LocalLLaMA/comments/1p82u5k/local_videototext_pipeline_on_apple_silicon/
 28. Donkey545/wyoming-faster-whisper-rocm - GitHub, accessed July 6, 2026,
<https://github.com/Donkey545/wyoming-faster-whisper-rocm>
 29. Day 0 Support for Gemma 4 on AMD Processors and GPUs, accessed July 6, 2026,
<https://www.amd.com/en/developer/resources/technical-articles/2026/day-0-support-for-gemma-4-on-amd-processors-and-gpus.html>
 30. AMD GPUs for AI workloads (LLMs, Diffusion, and Tools) - Hugging Face, accessed July 6, 2026,
https://huggingface.co/datasets/John6666/knowledge_base_md_for_rag_1/blob/24fa6877b2779b087c622f529dce27cabb435e03/amd_gpu_ai_20251121.md
 31. Best practices for competitive inference optimization on AMD Instinct™ MI300X GPUs, accessed July 6, 2026,
https://rocm.blogs.amd.com/artificial-intelligence/LLM_Inference/README.html
 32. AMD Instinct MI300X workload optimization - ROCm Documentation, accessed July 6, 2026,
<https://rocm.docs.amd.com/en/docs-6.2.2/how-to/tuning-guides/mi300x/workload>

- [d.html](#)
33. AMD RX 7900 XTX + ROCm + Gemma 4 26B — here's what actually worked for me - Reddit, accessed July 6, 2026, https://www.reddit.com/r/ROCm/comments/1tbu8m9/amd_rx_7900_xtx_rocm_gemma_4_26b_heres_what/
 34. AMD GPUs - llm-tracker, accessed July 6, 2026, <https://llm-tracker.info/howto/AMD-GPUs>
 35. Dealing with LLM sycophancy (alignment tax): How do you write system prompts for constructive criticism? : r/LocalLLaMA - Reddit, accessed July 6, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1rr6iuo/dealing_with_llm_sycophancy_alignment_tax_how_do/
 36. TL;DR: A better way to prompt LLMs (inspired by Andrej Karpathy) : r/BypassAiDetect, accessed July 6, 2026, https://www.reddit.com/r/BypassAiDetect/comments/1pi761f/tldr_a_better_way_to_prompt_llms_inspired_by/
 37. Context-Aware Pragmatic Metacognitive Prompting for Sarcasm Detection - arXiv, accessed July 6, 2026, <https://arxiv.org/html/2511.21066v1>
 38. Which LLM model(s) is the funniest one? As in, the one with the best comedic writing skills. : r/LocalLLaMA - Reddit, accessed July 6, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1furllv/which_llm_models_is_the_funniest_one_as_in_the/
 39. 10 Examples of Tone-Adjusted Prompts for LLMs - Latitude.so, accessed July 6, 2026, <https://latitude.so/blog/10-examples-of-tone-adjusted-prompts-for-llms>
 40. History repeats : r/programminghumor - Reddit, accessed July 6, 2026, https://www.reddit.com/r/programminghumor/comments/1u3ybnf/history_repeats/
 41. thankYouLLM : r/ProgrammerHumor - Reddit, accessed July 6, 2026, <https://www.reddit.com/r/ProgrammerHumor/comments/1rw777b/thankyoullm/>
 42. UB Forte - Hilarious Programming Humor - Chris Oldwood - ACCU 2025 Short Talks, accessed July 6, 2026, <https://www.youtube.com/watch?v=hBYWiQfG4Gs>
 43. AI Humor - NeuroLogica Blog, accessed July 6, 2026, <https://theness.com/neurologicablog/ai-humor/>
 44. AMD Instinct MI300 Series / MI350 Series workload optimization - ROCm Documentation, accessed July 6, 2026, <https://rocm.docs.amd.com/en/latest/how-to/rocm-for-ai/inference-optimization/workload.html>
 45. powderluv/vllm-docs: Documentation for vLLM Dev Channel releases - GitHub, accessed July 6, 2026, <https://github.com/powderluv/vllm-docs>
 46. vLLM V1 performance optimization - ROCm Documentation - AMD, accessed July 6, 2026, <https://rocm.docs.amd.com/en/latest/how-to/rocm-for-ai/inference-optimization/vllm-optimization.html>
 47. Fastest LLM Inference APIs in 2026: TTFT and Throughput Guide - Inworld AI, accessed July 6, 2026, <https://inworld.ai/resources/fastest-llm-inference-api>
 48. Local LLM Video Captioning: Private, Powerful, Open-Source | BuildZn, accessed

July 6, 2026,

<https://www.buildzn.com/blog/local-llm-video-captioning-private-powerful-open-source>

49. Video Subtitle Generation using Whisper and OpenVINO, accessed July 6, 2026, <https://docs.openvino.ai/2024/notebooks/whisper-subtitles-generation-with-output.html>
50. Chapter-Llama: Efficient Chaptering in Hour-Long Videos with LLMs - arXiv, accessed July 6, 2026, <https://arxiv.org/html/2504.00072v1>
51. generative-ai/gemini/use-cases/multimodal-data-curation/captioning.ipynb at main - GitHub, accessed July 6, 2026, <https://github.com/GoogleCloudPlatform/generative-ai/blob/main/gemini/use-cases/multimodal-data-curation/captioning.ipynb>
52. Explicit Temporal-Semantic Modeling for Dense Video Captioning via Context-Aware Cross-Modal Interaction - arXiv, accessed July 6, 2026, <https://arxiv.org/html/2511.10134v1>